

0.87.7. La fonction couleurTexte() — la couleur des lettres

`#include <couleurTexte>` : `couleurTexte(rouge, vert, bleu)` colore toutes les lettres d'un coup (Game Boy Color).

1. Ce qu'il faut comprendre

Ce qu'elle fait : `couleurTexte(rouge, vert, bleu)` change la couleur de **toutes les lettres** de l'écran. C'est `couleurFond(0, 3, rouge, vert, bleu)` : les lettres sont dessinées dans la teinte 3, et les cases dans la palette 0.

La ligne à écrire : `#include <couleurTexte>`.

Ses arguments : rouge, vert, bleu, de 0 à 31.

Ce qu'elle coûte : environ **80 octets** la première fois.

Essaie : `couleurTexte(0, 20, 0);` : des lettres vertes.

On retrouve couleurTexte dans 24 leçons

[0.88](#), [0.88.1](#), [0.88.2](#), [0.90](#), [0.90.1](#), [0.90.2](#), [0.90.4](#), [0.90.5](#), [0.90.6](#), [0.90.7](#), [0.90.8](#), [0.91](#), [0.91.1](#), [0.91.2](#), [0.91.3](#), [0.91.4](#), [0.91.5](#), [0.91.6](#), [0.91.7](#), [0.91.8](#), [0.91.9](#), [0.92](#), [0.92.1](#), [0.92.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **380 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
main()         41 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
CopierTuiles   58 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         128 o ← CopierTuiles — 8 lettres sur 43
"TOUT EST"     8 o ← texte() — un texte gravé
"EN ROUGE"     8 o ← texte() — un texte gravé
```

Lettres gravées : EGNORSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte()      × 2    22 o sur place, dans main() + EcrireTexte (10 o)
```

```
couleurFond() × 1    12 o sur place, dans main()
image()      × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <couleurTexte> : la couleur des lettres ----

#include <couleurTexte> // couleurTexte() : toutes les lettres d'une couleur
#include <texte>
```

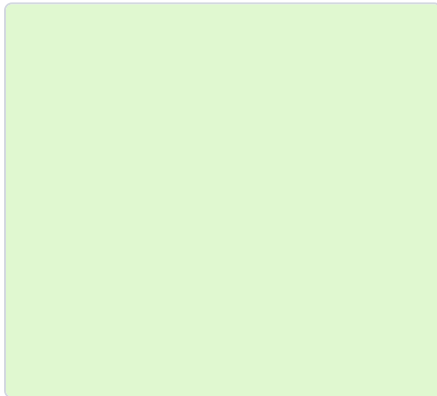
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    couleurTexte(31, 0, 0); // rouge 31, vert 0, bleu 0 : du rouge
    texte(4, 6, "TOUT EST");
    texte(4, 8, "EN ROUGE");

    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Deux lignes de texte en rouge.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la cartouche tourne en couleur
- ✓ la teinte 3 de la palette 0 est rouge (31)
- ✓ le texte est écrit

